



UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

Departamento Académico de Informática
Escuela Profesional: Ing. Informática y de Sistemas

COMPUTACIÓN GRÁFICA I

Práctica N°11

TRANSFORMACIONES AVANZADAS EN 3D

Reflexión general y distorsión dinámica

Estudiante: Efrain Vitorino Marin

Código: 160337

Profesor: Dr. Hans Harley Ccacyahuilca Bejar

Repositorio: <https://github.com/lovito99/laboGrafica/tree/main/lab011>

Resumen

En esta práctica se desarrollaron tres transformaciones avanzadas en el espacio tridimensional usando OpenGL moderno. Se implementó la reflexión de una pirámide sobre el plano arbitrario $3x + 5y + 6z = 0$, la reflexión estricta sobre el plano infinito $Z = 5$ y una distorsión dinámica cruzada aplicada sobre la pirámide reflejada.

Objetivos

- Implementar reflexiones tridimensionales mediante matrices homogéneas de 4×4 .
- Aplicar una reflexión general respecto a un plano arbitrario que pasa por el origen.
- Deducir el término de traslación para reflejar respecto al plano $Z = 5$.
- Construir una matriz de cizallamiento 3D con factores dependientes del tiempo.
- Visualizar los resultados con OpenGL moderno, GLFW, GLEW y GLM.

Librerías usadas

La implementación mantiene el estilo de la plantilla del PDF, usando OpenGL moderno, shaders y matrices enviadas como variables uniformes.

Listing 1: Librerías principales usadas.

```
1 #include <GL/glew.h>
2 #include <GLFW/glfw3.h>
3 #include <glm/glm.hpp>
4 #include <glm/gtc/matrix_transform.hpp>
5 #include <glm/gtc/type_ptr.hpp>
```

Shaders

El *vertex shader* recibe la posición de cada vértice y aplica las matrices **projection**, **view** y **transformMatrix**. El *fragment shader* colorea cada objeto mediante una variable uniforme.

Listing 2: Vertex shader usado en la practica.

```
1 const char* const shaderVert = R"glsl(
2 #version 330 core
3 layout (location = 0) in vec3 aPos;
4 uniform mat4 transformMatrix;
5 uniform mat4 view;
6 uniform mat4 projection;
```

```
7
8 void main()
9 {
10     gl_Position = projection * view * transformMatrix * vec4(aPos, 1.0);
11 }
12 )glsl";
```

Listing 3: Fragment shader usado para colorear la piramide y los ejes.

```
1 const char* const shaderFrag = R"glsl(
2 #version 330 core
3 out vec4 FragColor;
4 uniform vec3 objectColor;
5
6 void main()
7 {
8     FragColor = vec4(objectColor, 1.0);
9 }
10 )glsl";
```

Geometría de la escena

La escena contiene ejes coordenados y una pirámide definida con cuatro vértices. La pirámide se dibuja mediante aristas para poder comparar con claridad la figura original y la figura transformada.

Listing 4: Vertices de la piramide usada en las tres actividades.

```
1 const float pyramidVertices[] = {
2     40.0f, 20.0f, 40.0f,
3     60.0f, 30.0f, 20.0f,
4     40.0f, 50.0f, 30.0f,
5     20.0f, 30.0f, 20.0f
6 };
7
8 const unsigned int pyramidEdges[] = {
9     0, 1, 1, 2, 2, 0,
10    0, 3, 1, 3, 2, 3
11 };
```

Cámara y proyección

Para observar las transformaciones en 3D se usa una proyección ortográfica y una cámara construida con `glm::lookAt`.

Listing 5: Envío de matrices de cámara al shader.

```

1 inline void sendCamera(GLuint program)
2 {
3     const glm::mat4 projection = glm::ortho(
4         -120.0f, 120.0f,
5         -100.0f, 130.0f,
6         -250.0f, 250.0f);
7
8     const glm::mat4 view = glm::lookAt(
9         glm::vec3(95.0f, 80.0f, 145.0f),
10        glm::vec3(25.0f, 25.0f, 25.0f),
11        glm::vec3(0.0f, 1.0f, 0.0f));
12
13    glUniformMatrix4fv(glGetUniformLocation(program, "projection"),
14                        1, GL_FALSE, glm::value_ptr(projection));
15    glUniformMatrix4fv(glGetUniformLocation(program, "view"),
16                        1, GL_FALSE, glm::value_ptr(view));
17 }

```

Actividad A: Reflexión sobre $3x + 5y + 6z = 0$

El plano pasa por el origen y su vector normal es:

$$\vec{n} = (3, 5, 6)$$

Para reflejar un punto p respecto a un plano que pasa por el origen se usa:

$$p' = p - 2\vec{n} \frac{\vec{n} \cdot p}{\vec{n} \cdot \vec{n}}$$

En forma matricial:

$$R = I - 2 \frac{\vec{n}\vec{n}^T}{\vec{n} \cdot \vec{n}}$$

Como:

$$\vec{n} \cdot \vec{n} = 3^2 + 5^2 + 6^2 = 70$$

la matriz queda:

$$R = \begin{pmatrix} 1 - \frac{18}{70} & -\frac{30}{70} & -\frac{36}{70} & 0 \\ -\frac{30}{70} & 1 - \frac{50}{70} & -\frac{60}{70} & 0 \\ -\frac{36}{70} & -\frac{60}{70} & 1 - \frac{72}{70} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Listing 6: Construcción de la reflexión sobre un plano que pasa por el origen.

```
1 inline void createReflectionPlaneThroughOrigin(float* mat, float a, float b,  
2 float c)  
3 {  
4     createIdentityMatrix(mat);  
5  
6     const float denominator = a * a + b * b + c * c;  
7     if (denominator == 0.0f) {  
8         return;  
9     }  
10  
11     const float n[3] = {a, b, c};  
12     for (int row = 0; row < 3; ++row) {  
13         for (int col = 0; col < 3; ++col) {  
14             const float identity = row == col ? 1.0f : 0.0f;  
15             mat[col * 4 + row] =  
16                 identity - (2.0f * n[row] * n[col]) / denominator;  
17         }  
18     }
```

Listing 7: Uso de la matriz en la actividad A.

```
1 float identity[16];  
2 float reflection[16];  
3 createIdentityMatrix(identity);  
4 createReflectionPlaneThroughOrigin(reflection, 3.0f, 5.0f, 6.0f);  
5  
6 drawPyramid(buffers, transformLoc, colorLoc, identity, 0.90f, 0.12f, 0.10f);  
7 drawPyramid(buffers, transformLoc, colorLoc, reflection, 0.05f, 0.52f, 0.85f);
```

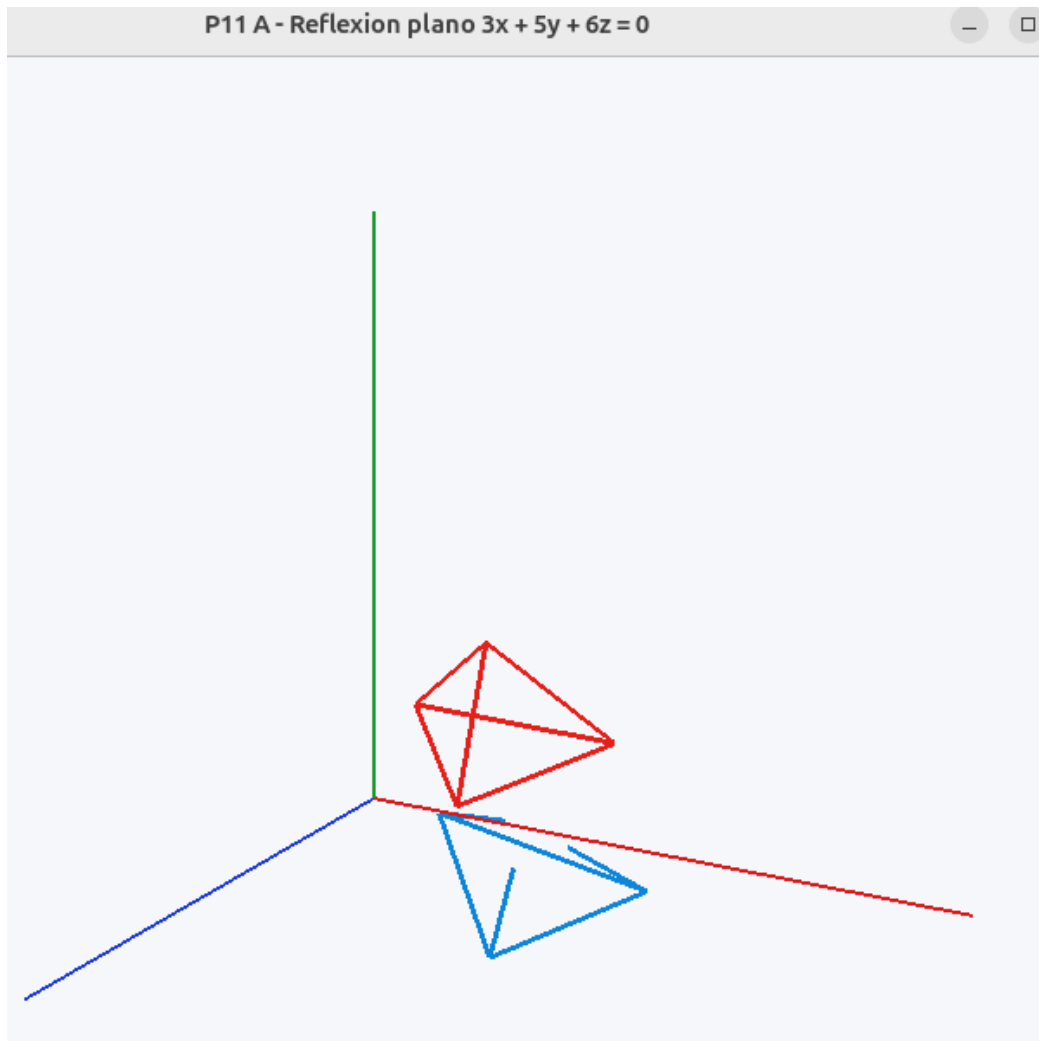


Figura 1: Reflexión de la pirámide sobre el plano $3x + 5y + 6z = 0$.

Actividad B: Reflexión estricta sobre $Z = 5$

Para reflejar respecto al plano infinito $Z = 5$, se traslada el plano hacia el origen, se refleja sobre $Z = 0$ y se retorna a la posición original:

$$M = T(0, 0, 5) R_{xy} T(0, 0, -5)$$

La transformación resultante es:

$$x' = x, \quad y' = y, \quad z' = 10 - z$$

La matriz homogénea queda:

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 10 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

El valor 10 corresponde al término de traslación acoplado $2 \cdot 5$.

Listing 8: Reflexion sobre el plano Z igual a una constante.

```
1 inline void createReflectionZEquals(float* mat, float zPlane)
2 {
3     createIdentityMatrix(mat);
4     mat[10] = -1.0f;
5     mat[14] = 2.0f * zPlane;
6 }
```

Listing 9: Uso de la reflexion estricta sobre Z=5.

```
1 float identity[16];
2 float reflectionZ5[16];
3 createIdentityMatrix(identity);
4 createReflectionZEquals(reflectionZ5, 5.0f);
5
6 drawPyramid(buffers, transformLoc, colorLoc, identity, 0.90f, 0.12f, 0.10f);
7 drawPyramid(buffers, transformLoc, colorLoc, reflectionZ5, 0.06f, 0.58f, 0.20f);
```

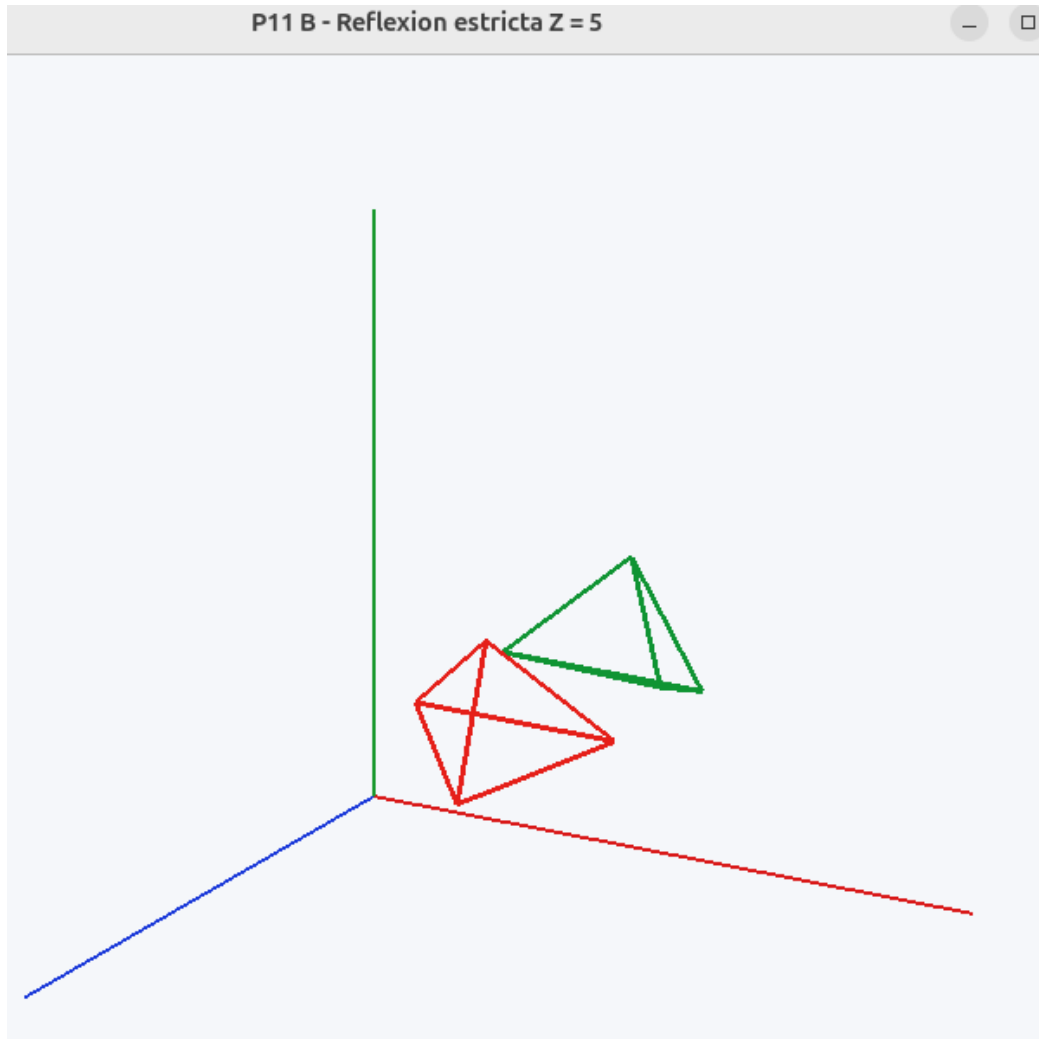


Figura 2: Reflexión geométrica estricta de la pirámide sobre el plano $Z = 5$.

Actividad C: Distorsión dinámica cruzada

La actividad final aplica cizallamiento 3D sobre la pirámide reflejada. La matriz general usada es:

$$Sh = \begin{pmatrix} 1 & shxy & shxz & 0 \\ shyx & 1 & shyz & 0 \\ shzx & shzy & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

En esta práctica se solicita:

$$shxy = \sin(t), \quad shyz = \cos(t)$$

Por tanto:

$$x' = x + \sin(t)y$$

$$y' = y + \cos(t)z$$

$$z' = z$$

Listing 10: Construcción de matriz de shearing 3D.

```

1 inline void createShearing3D(float* mat, float shxy, float shxz,
2                               float shyx, float shyz,
3                               float shzx, float shzy)
4 {
5     createIdentityMatrix(mat);
6     mat[4] = shxy;
7     mat[8] = shxz;
8     mat[1] = shyx;
9     mat[9] = shyz;
10    mat[2] = shzx;
11    mat[6] = shzy;
12 }

```

Listing 11: Distorsión dinámica aplicada sobre la pirámide reflejada.

```

1 const float time = static_cast<float>(glfwGetTime());
2 float shear[16];
3 float dynamicTransform[16];
4
5 createShearing3D(
6     shear,
7     std::sin(time), 0.0f,
8     0.0f, std::cos(time),
9     0.0f, 0.0f);
10
11 multiplyMatrices(shear, reflectionZ5, dynamicTransform);
12
13 drawPyramid(buffers, transformLoc, colorLoc, identity, 0.72f, 0.72f, 0.72f);
14 drawPyramid(buffers, transformLoc, colorLoc, reflectionZ5, 0.06f, 0.58f, 0.20f);
15 drawPyramid(buffers, transformLoc, colorLoc, dynamicTransform, 0.88f, 0.28f,
16             0.06f);

```

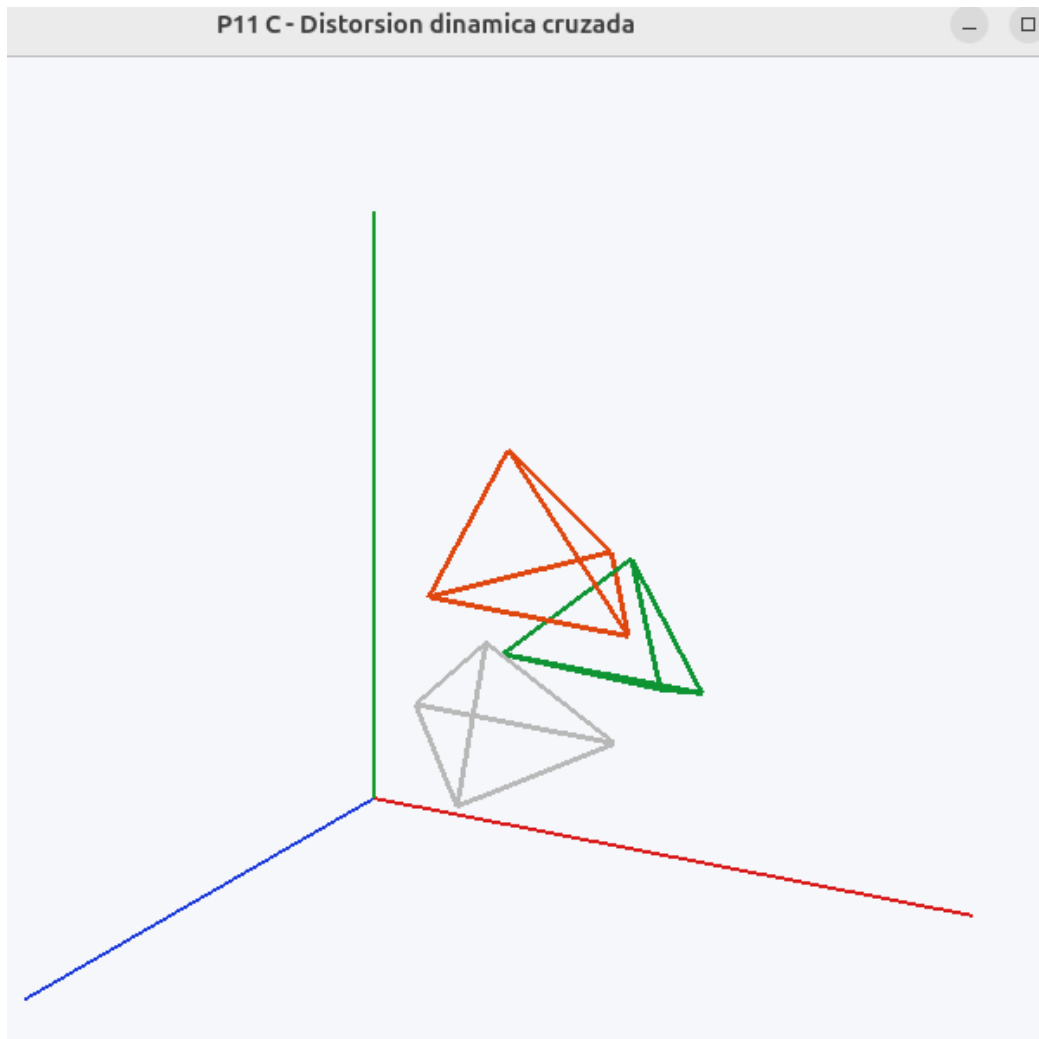


Figura 3: Distorsión dinámica cruzada aplicada sobre la pirámide reflejada.

Compilación y ejecución

Los tres programas se compilan con CMake:

Listing 12: Comandos de compilacion.

```
1 cd ~/Documentos/GitHub/laboGrafica/lab011
2 mkdir -p build
3 cd build
4 cmake .. -G "Unix Makefiles"
5 make
```

Luego se ejecutan:

Listing 13: Comandos de ejecucion.

```
1 ./actARefPlano
2 ./actBRefZ5
3 ./actCShearDyn
```

Conclusiones

- La reflexión sobre el plano $3x + 5y + 6z = 0$ se obtuvo con una matriz general basada en el vector normal del plano.
- La reflexión sobre $Z = 5$ requiere incorporar el término de traslación 10 dentro de la matriz homogénea.
- La distorsión dinámica cruzada se logró modificando simultáneamente las componentes X e Y con factores dependientes del tiempo.
- El uso de shaders y matrices uniformes permite separar la geometría original de las transformaciones aplicadas.